Flash-Sale Backend: Performance Comparison

From a naive purchase endpoint to a Redis-gated, BullMQ-backed design

Project: flashsalessys (Node.js, Express, MongoDB, Redis, BullMQ) | Load tool: k6 | Report date: 23 October 2026

1. Executive summary

We built the purchase flow in four stages and load-tested each one with the same k6 scenario: 30 simultaneous users competing for a product with only 5 units in stock. The goal was to understand what each stage actually fixes, and how much evidence we have for each improvement.

10 / 5	5 / 5	25 ms	0 errors
units sold from 5 in stock by the naive version (oversold)	units sold by every later stage (no overselling)	median with Redis gate, vs 53 to 128 ms for atomic MongoDB	900-request BullMQ run, p95 107 ms

Key findings

- **Correctness was the biggest win.** The naive read, check, save endpoint sold 10 units of a 5-unit product under concurrent load. A single atomic MongoDB update fixed this, and every later stage sold exactly 5.
- **The Redis gate cut median latency.** Its median was 24.8 ms, compared with 53.5 to 128.3 ms across three atomic MongoDB runs. That is outside the run-to-run spread we observed. The p95 (455 ms vs 474 to 790 ms) and throughput differences are small enough to be within noise.
- **The BullMQ stage was stable at higher load.** It handled 900 requests with 0 errors and a p95 of 107 ms. Accepted purchases took about as long as rejections (median 51 ms vs 47 ms), which suggests the MongoDB write is no longer in the request path. This run used 10x more requests than the others, so it is not a like-for-like latency comparison (see section 6).
- **Evidence gaps remain.** The runs are short and on one machine, and the supplied results do not include the worker logs or final MongoDB stock for the queue run. Section 7 lists a re-test plan.

At a glance

Stage	Units sold (stock 5)	Median	p95	Req/s	Reading
1. Naive	**10**	78 ms	709 ms	89	Oversells; baseline
2. Atomic MongoDB (3 runs)	5	53 to 128 ms	474 to 790 ms	86 to 135	Correct; latency varies a lot between runs
3. Redis gate	5	25 ms	455 ms	131	Correct; median clearly lower
4. Redis + BullMQ (900 requests)	5	47 ms	107 ms	180	Correct; different load, not directly comparable

2. Test setup

Every run used the same k6 script. Each run created a fresh product with 5 units, then 30 virtual users sent buy requests at the same moment. Because there are many more requests than units, most requests are expected to be sold-out rejections (85 of 90 in the 90-request runs).

Item	Value
Load tool	k6, scenario per-vu-iterations
Virtual users	30 simultaneous
Stock per run	5 units (fresh product each run)
Requests	Runs R1 to R5: 90 requests (30 users x 3 attempts), about 1 second each. Run R6: 900 requests (30 x 30), about 5 seconds
Environment	Local development machine, Docker Compose: Node.js 20 (Express 5, Mongoose 9, ioredis 6), MongoDB community server, Redis Stack. k6, the app and both databases share the same machine
Pass/fail check	Threshold: successful purchases must never exceed 5
Metrics read	http_req_duration (median, average, p90, p95, max), requests per second, plus custom counters for successes and sold-out responses

The four stages

Stage	Design	What it is meant to fix
1. Naive	Read the product, check quantity in Node.js, subtract, then save()	Baseline. Four separate steps, so concurrent requests can read the same stale stock
2. Atomic MongoDB	One findOneAndUpdate with filter quantity at least 1 and an $inc of -1. Refined version also deletes cache keys instead of overwriting them and separates 404 from 409	Overselling. MongoDB checks and subtracts in one step
3. Redis stock gate	A Lua script atomically checks and decrements a stock:id counter in Redis before MongoDB is touched	Sold-out requests are answered by Redis alone and never reach MongoDB
4. Redis + BullMQ	After the Redis check, a job is added to a purchases queue and the API returns 202. A worker updates MongoDB (concurrency 5, 3 attempts, exponential backoff) and refunds Redis stock on final failure	Accepted purchases no longer wait for the MongoDB write; spikes are smoothed

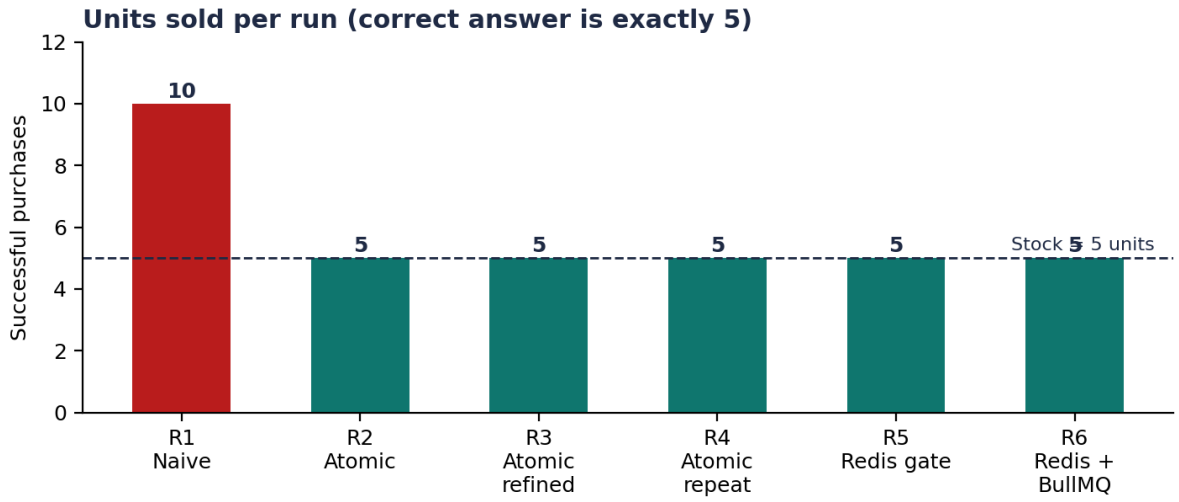
3. Raw results

All values come directly from the k6 summaries. Times are http_req_duration in milliseconds.

Run	Stage	Req	Sold	Median	Avg	p90	p95	Max	Req/s
R1	Naive (read, check, save)	90	**10**	78.5	200.4	628.1	708.9	792	89.1
R2	Atomic MongoDB, first version	90	5	128.3	227.2	646.9	789.9	857	86.2
R3	Atomic MongoDB, refined	90	5	53.5	126.3	389.7	474.0	531	134.8
R4	Atomic MongoDB, refined (repeat)	90	5	72.4	171.4	542.9	643.5	722	103.2
R5	Redis stock gate	90	5	24.8	137.5	430.0	454.9	489	130.7
R6	Redis gate + BullMQ queue	900	5	47.4	55.4	92.6	106.7	499	179.6

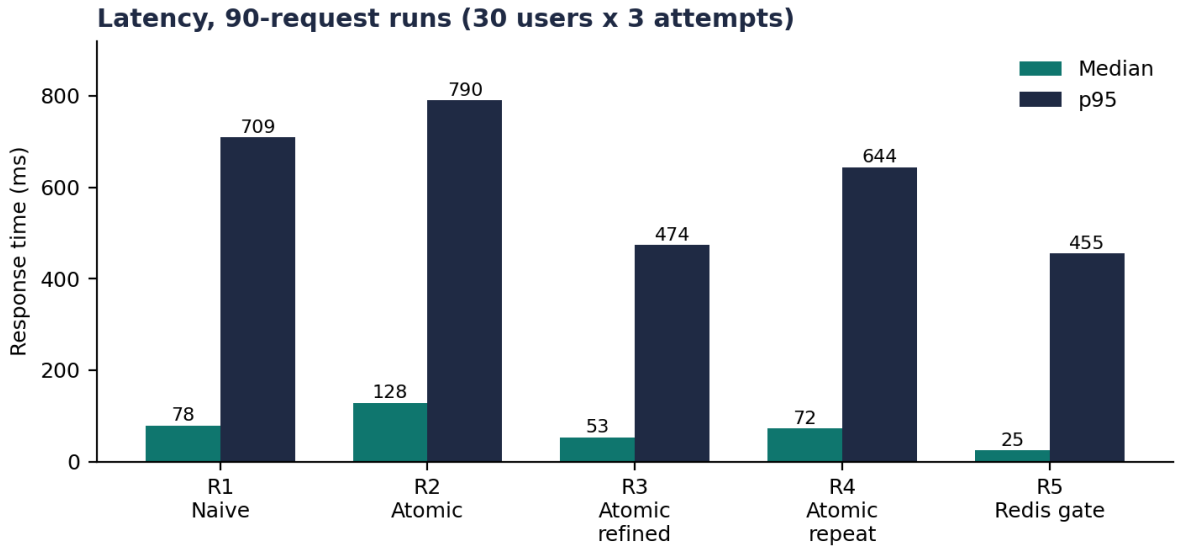
R6 (shaded) ran 10 times as many requests as the others, so compare it with them cautiously. R2 used an earlier version of the endpoint that overwrote the cache with set() and returned 400 for sold-out; R3 and R4 ran the refined code. Runs R1 to R3 also showed a 1.08% failed-request rate in k6; this was a script artifact (the product-creation call returns 201, which the script had not yet listed as expected) and was fixed afterwards.

3.1 Correctness



The naive endpoint sold 10 units from a stock of 5. In that run, 10 requests read a quantity above zero before any of them saved, so all of them passed the check. All later stages sold exactly 5 in every run, and the sold-out responses add up to the remaining requests (80 + 10 = 90 for R1; 85 + 5 = 90 for R2 to R5; 895 + 5 = 900 for R6).

4. Latency across the 90-request runs



Three runs of the atomic MongoDB endpoint produced medians of 128, 53 and 72 ms, and p95 values of 790, 474 and 644 ms. That spread is the natural noise of a one-second test on a shared laptop, and it limits how much we can read into any single run. The naive stage (78 ms median) sits inside this spread, so we cannot say the atomic update is faster or slower than it; what we can say is that it is correct.

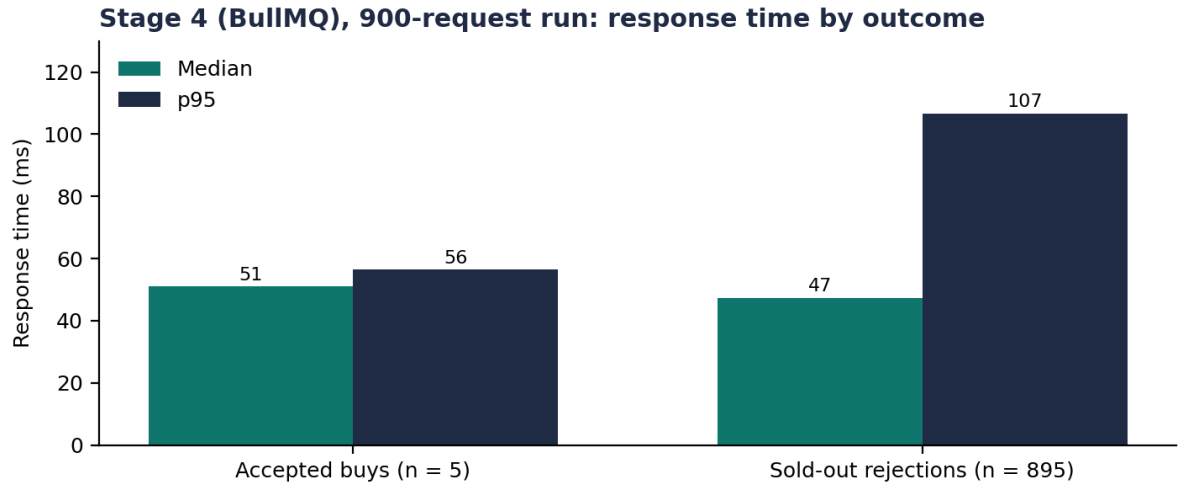
4.1 Stage 3 against Stage 2

Metric	Stage 2 average of 3 runs	Stage 2 best run	Stage 3 (R5)	Change vs average	Change vs best
Median	84.7 ms	53.5 ms	24.8 ms	-71%	-54%
p95	636 ms	474 ms	455 ms	-28%	-4%
Requests/s	108	135	131	+21%	-3%

Reading: the median is 71% lower than the Stage 2 average and 54% lower than even the best Stage 2 run, which is a result outside the observed spread. The p95 and throughput differences against the best Stage 2 run are only a few percent, so we treat them as unproven. This pattern is what the design predicts: 85 of 90 requests are rejections, and the Redis gate answers them without MongoDB (the fastest request in R5 took about 0.5 ms), which lowers the typical request. The slowest requests are dominated by the first burst of 30 simultaneous requests and by the 5 purchases that still wait for MongoDB, which this stage does not change.

5. Stage 4: BullMQ queue

With the queue, the API runs the Redis check, adds a job and returns 202 Accepted. The MongoDB update happens later in a separate worker process. This run also recorded response times separately for accepted and rejected requests.



Metric (R6, 900 requests)	Value
Total requests / throughput	903 HTTP requests, 179.6 requests/s
Accepted purchases (202)	5 of 5 units; response time average 47.3 ms, median 51.0 ms, p95 56.5 ms, max 57.5 ms
Sold-out rejections	895; response time average 55.5 ms, median 47.3 ms, p90 92.6 ms, p95 106.7 ms, max 499 ms
All requests	Average 55.4 ms, median 47.4 ms, p95 106.7 ms
Errors	0 failed requests, 0 unexpected errors; the 5-unit threshold passed

Reading: accepted purchases responded in about the same time as rejections, so the successful path no longer waits for MongoDB. That is the intended effect of the queue. Two cautions apply: there were only 5 accepted purchases, which is a small sample, and no earlier stage recorded accepted and rejected response times separately, so we cannot yet measure the improvement against Stage 3 on the accepted path.

6. How far the evidence goes

- **Short runs.** The 90-request runs lasted about one second and the 900-request run about five. Warm-up, connection setup and a busy laptop can move a median by 50 ms or more, as the three atomic runs show.
- **Different load in R6.** A longer run spreads the cold-start cost over more requests, which lowers p95 for any design. The p95 of 107 ms in R6 against 455 ms in R5 therefore cannot be credited to the queue.
- **Shared machine.** k6, Node.js, MongoDB and Redis compete for the same CPU, and file watching through Docker adds background load. Absolute numbers will differ in production; relative comparisons on the same machine are more meaningful.
- **Inferred route labels.** The endpoint behind runs R4 and R6 was inferred from the conversation (atomic MongoDB route for R4, the default queue route for R6) and should be confirmed against the commands that were run.
- **No end-to-end check for the queue.** The supplied data show that 5 jobs were accepted, but not that the worker completed them. The final MongoDB stock, the Redis counter and the worker logs were not part of the data.
- **Rejections dominate.** With 5 units and 90 to 900 requests, 94% to 99% of requests are rejections. The tests therefore say much more about the rejection path than about the purchase path.

7. Conclusions and next steps

Conclusions. The atomic update is the change that makes the system correct. The Redis gate lowers the typical response time when most traffic is sold-out requests. The queue removes the database write from the purchase request and ran cleanly at 900 requests, though its latency benefit has not yet been measured under comparable conditions.

Recommended re-test, to compare the three routes like for like:

- Run the same k6 scenario (30 users x 30 attempts = 900 requests) against the atomic MongoDB, Redis gate and Redis + BullMQ routes, three runs each, and compare medians.
- Use a larger stock (for example 200 units) so that accepted purchases are numerous enough to compare their response time. Compare the purchase_duration metric across routes.
- After each queue run, wait a few seconds, then confirm that MongoDB stock and the Redis counter both reach the expected value, and that the worker logged one completed job per purchase.
- Test failure paths: stop MongoDB briefly during a run and confirm that retries and the Redis stock refund keep the counters consistent.

Hardening for production. Add an Order model with a unique job id so a retried job cannot subtract stock twice; make the stock-initialization endpoint a POST with admin authentication; and plan for the Redis stock counter being lost on a restart (re-initialize it from MongoDB).